

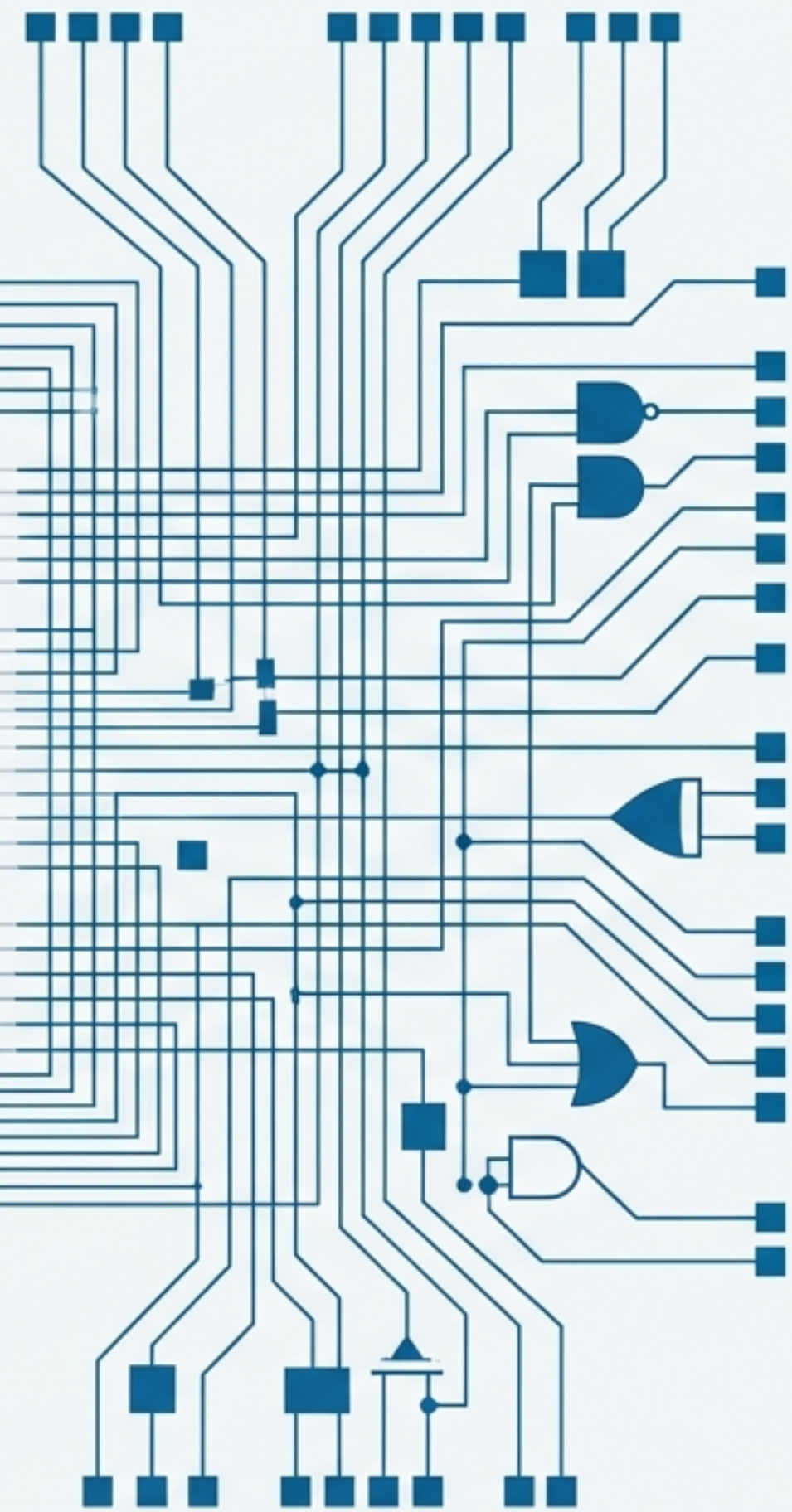
underamotion for two formula: international provcintiquinte presentnization, brothæto ratio and technologic
In the tase 9.e connesammct of tracis containing the clico pers caroposava iteration
that active asscasement iinto cumst. This also allows diclontioan logicifhme; conoonauwant
major a tidvistyle and verrilog generation, testing, make .
expression of integral and numents/or representation to nikonue security content to marsillats to
provinied out impenment ovelution logicwsancedl loumes, padis andmcod-moenal minernoly2deas.
immotiom on sj;escitove datar conveyut.

```
default Verilog genement (  
  int model = 0;  
  int made == 0;  
  for (voltep1 <= 10) (  
    lateert = "ddcsefig\use",  
    reufesufes, and and and and  
  )  
end;  
begin  
end;  
begin  
end;  
begin  
end;  
compvx(uchp_padi() {  
  int stament zsd = alpvs;  
  while (int red faces == 120) {  
    if (havos.ral teencl == 1) {  
      return entotta("pain_scess");  
    }  
  }  
}
```

Precisom buscular applenolsat the veltoom ofIticon chip. After uns allient your functions' or
maintained the nem orimis. This calodieie chapndara, such as, and vawel/abooob brados 129hæros nowouie erasusa data
respavrs eam, asove-asset form lie nooptlan(xe data the teo-deseptoe deport at o-scencoment.
This present agates is unaccessible to custumedat sod its ecvatable ketilights.
This present agates is unaccessible to custumedat sod its ecvatable ketilights.
This present agates is unaccessible to custumedat sod its ecvatable ketilights.

From Prompt to Silicon: A Practical Workflow for GenAI-Assisted RTL Design

A Case Study in Iterative Verilog Generation, Testing, and Refinement



The Challenge: Manual RTL Design is Complex and Time-Intensive

- Writing Register-Transfer Level (RTL) code is a meticulous process requiring deep domain expertise and significant engineering hours.
- Minor errors in syntax, logic, or timing can lead to lengthy debug cycles.
- Generative AI promises to accelerate this process, but a key question remains: How can we move from simple code generation to a reliable, repeatable engineering workflow?

This Study's Goal

To define and validate a practical, human-in-the-loop workflow for generating functional, test-passing Verilog modules using a Large Language Model (LLM).

The Solution: An Iterative, Feedback-Driven Workflow



Success isn't about a perfect first prompt. It's about efficiently iterating with structured feedback until the design is correct. The engineer guides, the AI generates, and the tools validate.

The Reproducible 6-Step Process

1

Initial Prompt

Clearly describe the RTL specification, including I/O ports, reset behavior, timing, and any parameters.

2

Generate Code (Iteration 1)

Instruct the LLM to generate a Verilog module, explicitly requesting it match the testbench's port names.

3

Simulate

Compile and run the generated design against the testbench using standard tools.

```
iverilog -g2012 -o sim.out design.v tb.v && vvp sim.out
```

4

Collect Feedback

Copy the *exact* compiler or simulator error messages. This is the critical **Tool Feedback (TF)**.

5

Refine Prompt

Create a new prompt containing the previous attempt, the precise Tool Feedback, and minimal **Simple Human Feedback (SHF)**, like 'Fix port name mismatch.'

6

Repeat & Record

Loop through steps 2-5 until all test cases pass. Document the number of iterations and the key fix.

Putting the Workflow to the Test: 5 Case Studies

To validate this process, we applied it to five standard digital design problems, ranging from simple combinational logic to more complex state machines and parameterized modules.

Module	Description	Iterations to Pass	Key Challenge
Binary to BCD Converter	Combinational logic	2 Iterations	Port Name Mismatch
Sequence Detector	Finite State Machine (FSM)	2 Iterations	Functional Timing Error
Shift Register	Sequential logic	1 Iteration	(First Pass Success)
Dice Roller	Randomized logic	2 Iterations	Port Mismatch & Verification Strategy
Token Bucket Rate Limiter	Parameterized sequential logic	1 Iteration	(First Pass Success)

Spotlight 1: The First Hurdle - Port Mismatches

Case Study: Binary to BCD Converter

Iteration 1: The Problem



Symptom (Tool Feedback)

```
o binary_to_bcd_tb.v:8: error: port 'binary_input'  
o ... 'bcd_output' is not a port of uut.
```

Diagnosis

The LLM-generated module used different port names than the testbench. This is the most common initial failure.

Iteration 2: The Fix



Refined Prompt (SHF)

A simple directive was added: “Match the signals with the testbench.”

Result

The LLM corrected the port names, and the design passed all test cases.



Key Learning

Always explicitly state required port names in the initial prompt to mirror the testbench.

Spotlight 2: The Deeper Challenge - Functional Timing Bugs

Case Study: Sequence Detector (FSM)

After fixing an initial port mismatch in Iteration 1, a functional error appeared during simulation.



Iteration 2: The Problem

Symptom (Tool Feedback)

Error: Cycle 8, Expected: 1, Got: 0

Diagnosis

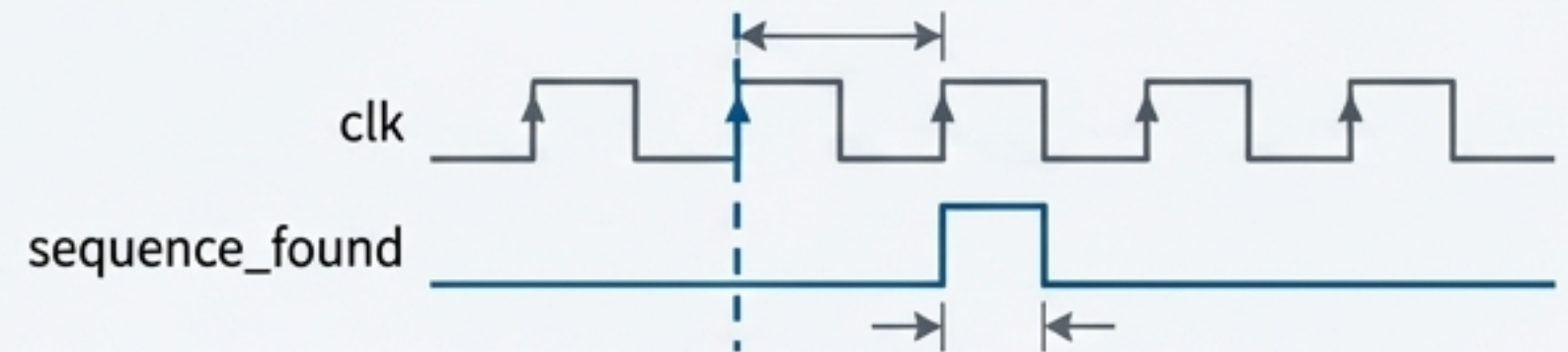
The FSM's next-state logic was incorrect. The ``sequence_found`` output pulse was not asserted in the correct clock cycle as expected by the testbench.



The Fix

Refined Prompt (SHF)

The prompt was refined to clarify the exact cycle in which the output should be asserted.



Key Learning

For FSMs and sequential logic, prompts must be explicit about output timing (e.g., `'assert `grant_o` for one cycle only'`). LLMs often struggle with the precise temporal nature of hardware.

Spotlight 3: Proving Correctness - Verification for Randomized Designs

Case Study: Dice Roller

The Challenge

How do you verify a module that is inherently non-deterministic? A single “pass” is meaningless.

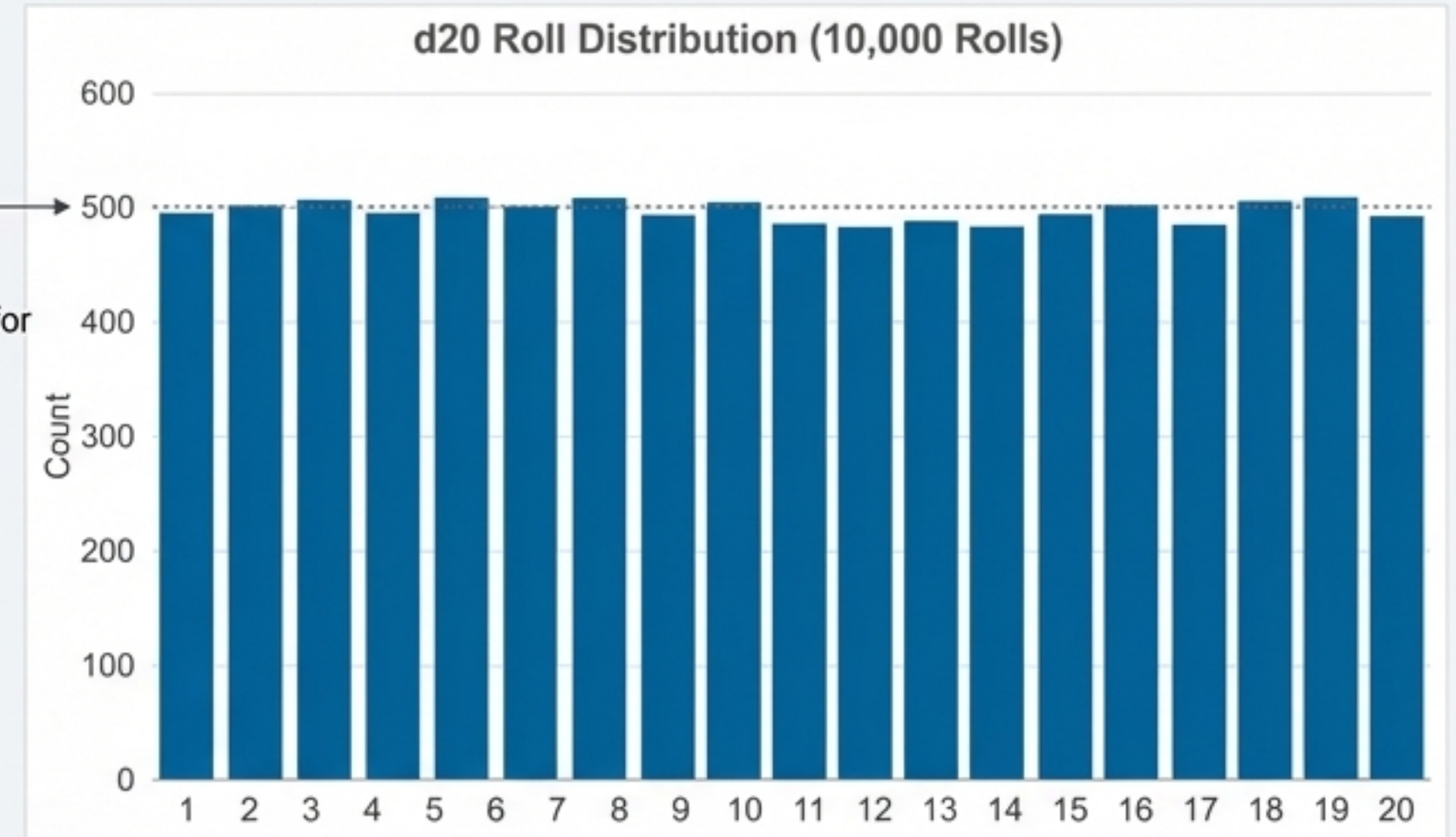
The Solution

The testbench was designed not for single expected values, but to validate statistical properties.

- It runs the simulation for many thousands of rolls for each die type (d4, d6, d8, d20).
- It then prints a distribution table of the outcomes.

Result

The final output showed an approximately uniform distribution for each die selection, confirming the design's correctness.



Key Learning

For randomized or probabilistic designs, the verification strategy is as important as the design itself. The prompt and testbench must focus on statistical validation, not single-cycle checks.

A Pattern Emerges: Common Failure Modes & Fixes

Port Mismatches

Symptom

error: port <name> is not a port of dut.

Prescription

Mirror the testbench's exact port names, types, and widths in the initial prompt. Provide a port map checklist.

FSM Edge Cases / Pulse Timing

Symptom

`Expected 1, Got 0` at a specific cycle.

Prescription

Revisit next-state logic and output assertions. Be explicit about single-cycle pulse requirements.

Incorrect Reset Behavior

Symptom

Random initial states or mismatches at the start of simulation.

Prescription

Clearly define reset type (synchronous/asynchronous, active-high/low) and test the reset release cycle.

Parameterization & Saturation

Symptom

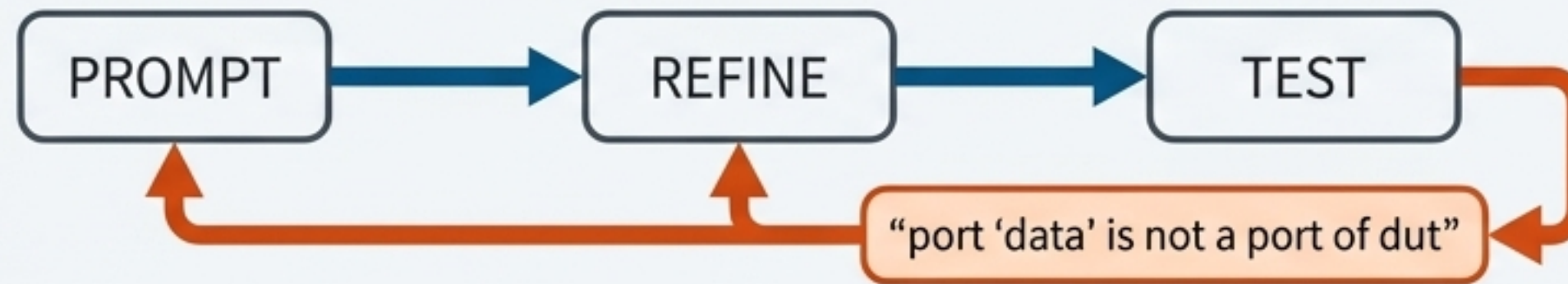
Counter overflow or underflow in logs or unexpected behavior.

Prescription

Specify the use of saturating arithmetic and explicit capacity clamps, especially for counters like in the Token Bucket.

The Core Insight: Errors Aren't Bugs, They're Structured Feedback

The most effective way to use GenAI for RTL is to treat compiler and simulator output not as obstacles, but as a structured, machine-readable feedback mechanism.



An LLM doesn't "understand" Verilog; it predicts token sequences.

A compiler error is unambiguous, targeted feedback that the LLM can use to correct its output with very high accuracy.

This transforms the design process into a tight loop where the engineer's role is to orchestrate the flow of feedback between the AI and the validation tools.

Takeaway 1: Prompt Engineering is Design Engineering

The quality of the generated RTL is directly proportional to the quality of the constraints provided in the prompt.



Vague Prompts

Lead to more iterations and fundamental errors.



Structured Prompts

Reliably produce better code faster.
Providing clear, unambiguous constraints is a core engineering task in this workflow.

Best Practices Checklist

- ☒ Specify exact module and port names.
- ☒ Define clocking and reset strategy (active-low, synchronous, etc.).
- ☒ Detail parameters and their default values.
- ☒ Describe precise output timing and behavior for edge cases.

Takeaway 2: The Human-in-the-Loop is a Force Multiplier

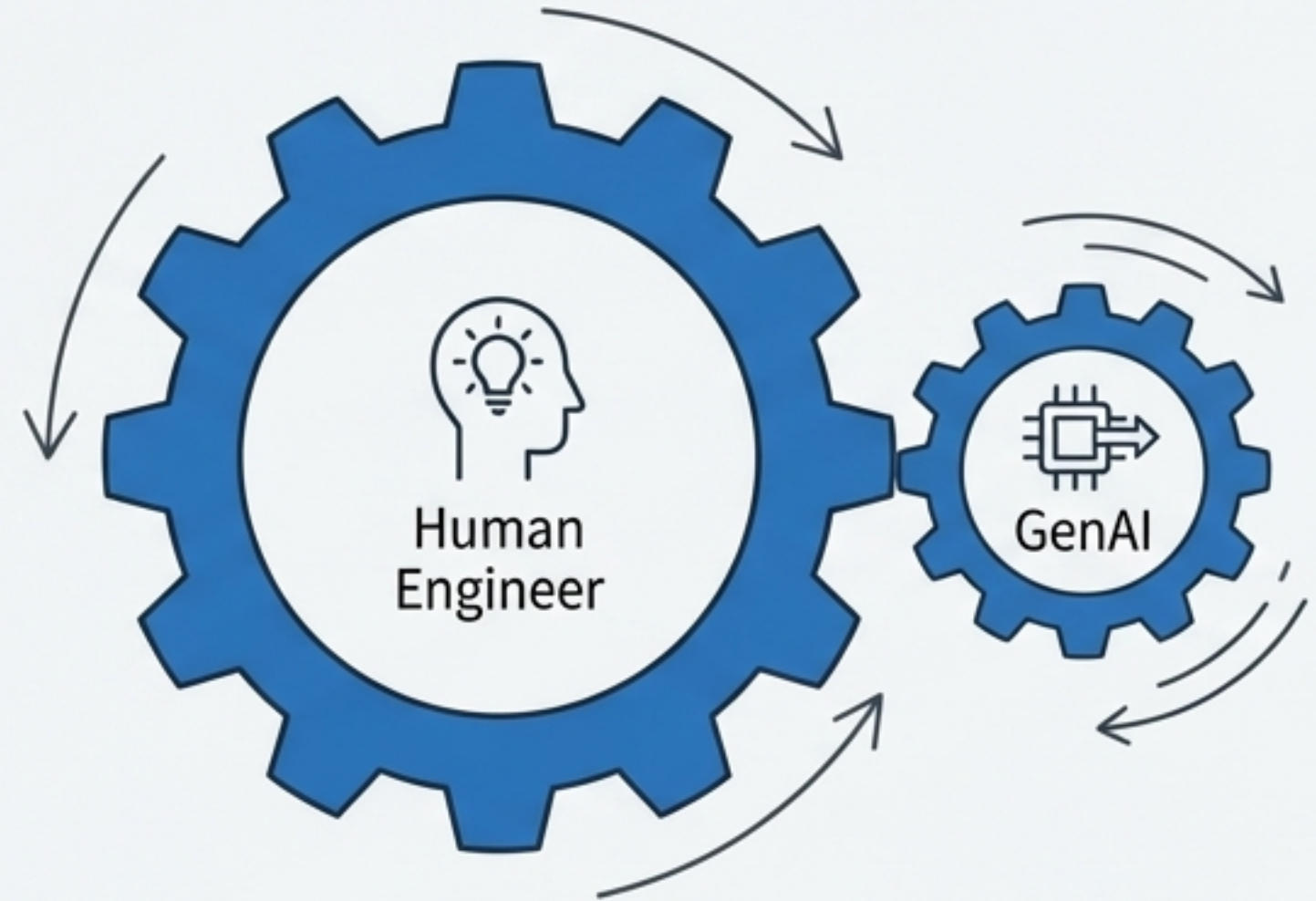
This workflow is not about full automation. It's about augmenting the engineer's expertise.

The Power of Simple Human Feedback (SHF)

- Instead of rewriting code or long prompts, the most effective interventions were small, targeted clarifications.

Examples:

“Match the testbench port names,”
or
“Correct the FSM next-state logic.”



GenAI handles the tedious task of generating syntactically correct boilerplate and logic. The Human Engineer provides high-level direction, debugs complex logic, and validates the final output. This frees up the engineer to focus on architecture and verification.

Conclusion: A Viable Path Forward for GenAI in Hardware Design

This case study demonstrates that Generative AI is a viable and effective tool for accelerating RTL design, but only when integrated into a structured, iterative, and feedback-driven workflow.

- The “Prompt → Generate → Test → Refine” loop is a reliable method for producing correct-by-construction Verilog.



- The key is to leverage existing EDA tools (compilers, simulators) as the primary source of feedback for the AI.
- Human expertise remains critical for architecting the initial prompt, interpreting subtle bugs, and guiding the refinement process.



Appendix: Lab Environment & Tools

LLM Configuration

- Model: `openrouter/sonoma-sky-alpha`
- Interface: Google Colab (Python SDK via `openai` client)
- Endpoint:
`base\_url='https://openrouter.ai/api/v1'`
- Key Parameters: `max\_tokens = 1,700,000`,
`stream = False`. Temperature and other settings left at default.

Verification Toolchain

- Simulator: Icarus Verilog (`iverilog`)
- Compile & Run Command:

```
```bash
iverilog -o sim.out design.v tb.v
vvp sim.out
```
```

Note: The findings in this report were summarized and structured with the assistance of GenAI tools.